ED1 03: Arrays e Listas Ligadas.

Fabio Irigon Pereira

Introdução

Array (Vila de Casas): Se você sabe o número da casa (índice), você chega nela instantaneamente (**O(1)**). Mas, se quiser construir uma casa nova no meio da rua, terá que empurrar todas as outras casas uma posição para o lado.

Lista Encadeada (Caça ao Tesouro): Para achar o terceiro "tesouro", você obrigatoriamente precisa ler a pista que está no primeiro, que te leva ao segundo, que te leva ao terceiro. É mais lento para encontrar (***O(n)***), mas para adicionar um novo tesouro, basta mudar a pista de um deles para apontar para o novo item.

Arrays

- Estrutura de dados contígua na memória.
- Acesso direto a elementos via índice (O(1)).
- Tamanho fixo (deve ser definido no momento da alocação).
- Ineficiente para inserções e remoções no meio do array (O(n)).

Arrays

```c
#include <stdio.h>

int main() {
   int arr[5] = {10, 20, 30, 40, 50}; // Array de tamanho fixo

   // Acesso direto ao terceiro elemento
   printf("Elemento na posição 2: %d\n", arr[2]); // O(1)

   return 0;
}
```

Listas Ligadas

- Conjunto de nós não contíguos armazenados **dinamicamente**.
- Cada nó contém:
  - **Dados**.
  - **Um ponteiro** para o próximo nó.
- Inserções e remoções são eficientes (O(1) se feitas no início).
- Acesso a um elemento específico é mais lento (O(n)) pois requer percorrer a lista.

Estrutura de um nó de lista ligada

```
typedef struct No {
   char nome[50];
   int idade;
   struct No * prox; // Ponteiro para o próximo nó
} No;
```

Arrays vs Listas

Operação	Array (Estático)	Lista Encadeada
Acesso por Índice	O(1) (Direto)	O(n) (Precisa percorrer)
Inserção no Início	O(n) (Precisa deslocar todos)	O(1) (Apenas troca ponteiros)
Inserção no Fim	O(1) (Se houver espaço)	O(1) (Se tiver ponteiro para o rabo)
Uso de Memória	Bloco contíguo fixo	Blocos dispersos (dinâmico) +1

Implementação: Arrays em C

Inserção em Arrays

```
#include < stdio.h >
   #include < string.h >

   #define TAMANHO_MAX 5

typedef struct {
   char nome[50];
   int idade;
} No;
```

Inserção em Arrays

```
int main() {
    No arr[TAMANHO_MAX];
    int tamanho = 0;

    // Inserindo elementos no array
    strcpy(arr[tamanho].nome, "Ana");
    arr[tamanho].idade = 25;
    tamanho++;

        ...
```

Inserção em Arrays

```
int main() {
    No arr[TAMANHO_MAX];
    int tamanho = 0;

    // Inserindo elementos no array
    strcpy(arr[tamanho].nome, "Ana");
    arr[tamanho].idade = 25;
    tamanho++;

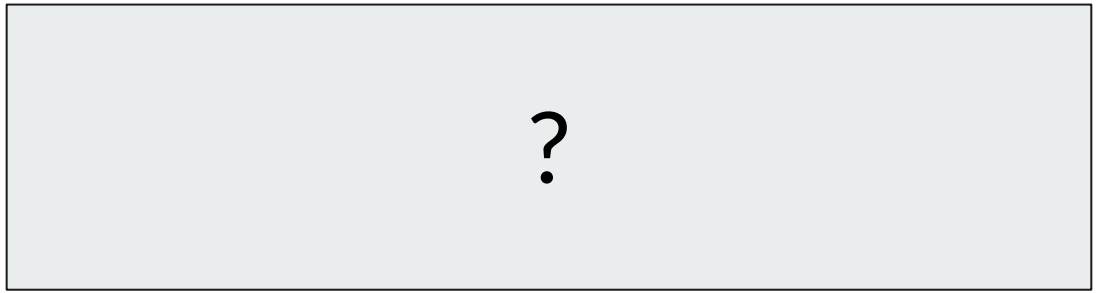
    strcpy(arr[tamanho].nome, "Carlos");
    arr[tamanho].idade = 30;
    tamanho++;

    // Imprimindo os elementos do array
    printf("Lista de Pessoas (Array):\n");
    for (int i = 0; i < tamanho; i++) {
        printf("Nome: %s, Idade: %d\n", arr[i].nome,
arr[i].idade);
    }

    return 0;
}
```

Inserção no meio do Array

```
void inserirMeio(No arr[], int tamanho, int pos, char nome[], int idade) {
    if (tamanho >= TAMANHO_MAX || pos > tamanho) {
        printf("Erro: Posição inválida ou array cheio.\n");
        return;
    }
```



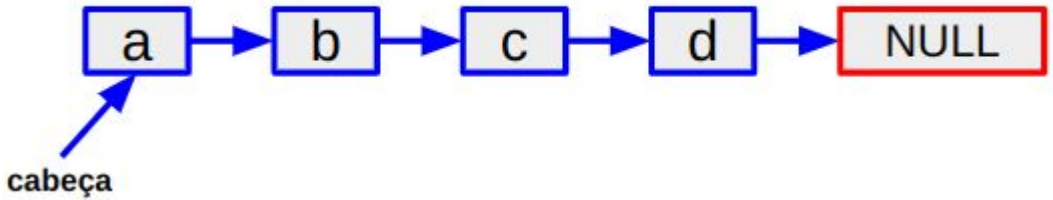

```
}
```

Exercício

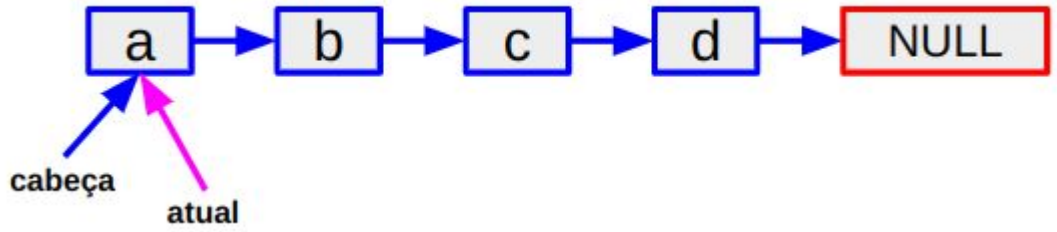
Tente criar uma função para remover um elemento no meio do array.

Implementação: Lista ligada

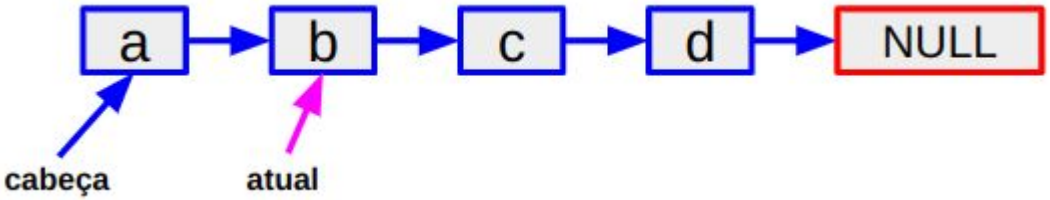
Imprimir (percorrer) lista ligada



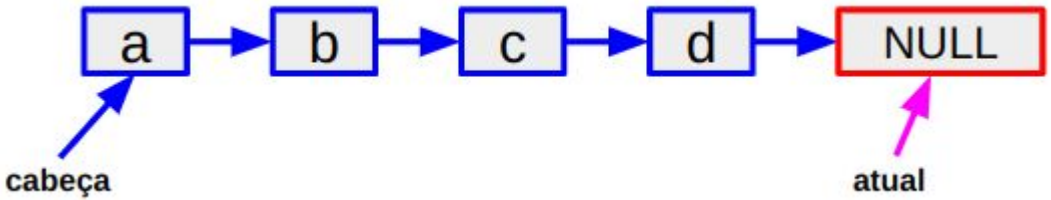
Imprimir (percorrer) lista ligada



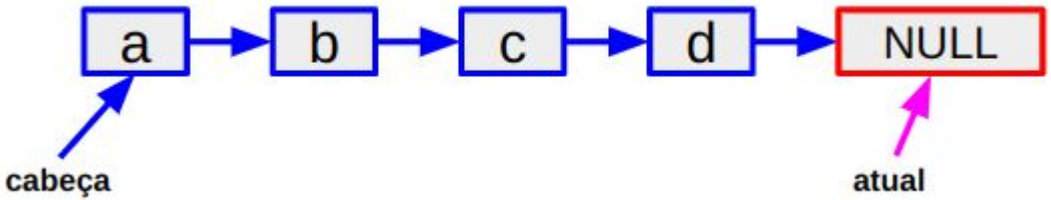
Imprimir (percorrer) lista ligada



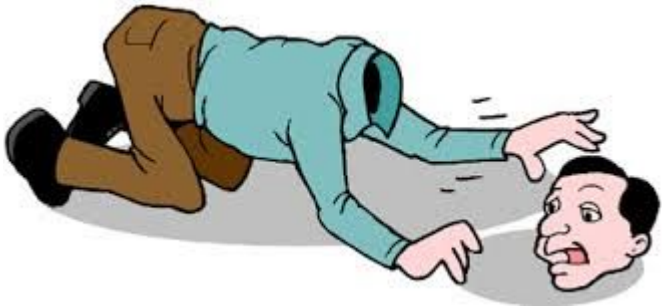
Imprimir (percorrer) lista ligada



Imprimir (percorrer) lista ligada



Atenção! **NUNCA PERCA A CABEÇA!!!**



Imprimir (percorrer) lista ligada

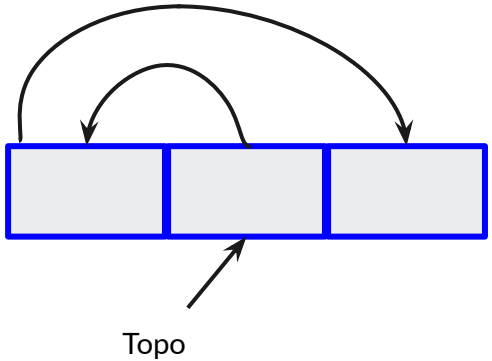
```c
// Função para imprimir a lista
void imprimirLista(No * cabeca) {
   No * atual = cabeca;
   printf("Lista de Pessoas (Lista Ligada):\n");
   while (atual != NULL) {
       printf("Nome: %s, Idade: %d\n", atual -> nome, atual -> idade);
       atual = atual -> prox;
   }
}
```

Exercício:

Tente criar uma rotina para adicionar um elemento entre o primeiro e o segundo elemento de uma lista ligada.

Exercício:

Crie um array com três nós do tipo Pessoa. Crie um ponteiro chamado topo apontando para o segundo elemento, faça o segundo apontar para o primeiro e o primeiro apontar para o terceiro. Percorra a lista encadeada imprimindo cada elemento.



Exercícios:

Crie 5 estruturas do tipo pessoa e ligue uma à outra.

Crie uma função que receba um ponteiro para o primeiro elemento, percorra a lista e imprima cada membro.

Crie uma função que receba um ponteiro para o primeiro elemento, e um inteiro ‘n’ e imprima o elemento da posição ‘n’.

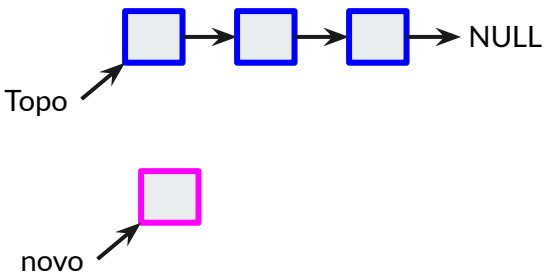
Crie uma função que remova o segundo elemento da lista, garantindo que o primeiro agora aponte para o terceiro.

Exercício:

Crie uma função que receba um ponteiro duplo para o início de uma fila e um ponteiro para um novo elemento e retorne a fila com o novo nó adicionado.

```
void add_member(Pessoa **topo, Pessoa *novo){...}
```

Antes:



Depois:

